

CESAR SCHOOL
CIÊNCIA DA COMPUTAÇÃO
TEORIA DA COMPUTAÇÃO

ALGORITMO A* (A-ESTRELA)

Análise teórica e experimental do caminho mínimo em grades:
implementações em Python e JavaScript

Arthur Padilha Pedro Tojal Arthur Lins Luis Guilherme

Prof. Daniel Bezerra

Recife

2026

SUMÁRIO

1	Introdução	3
2	O Algoritmo A*	4
2.1	Definição e objetivo	4
2.2	Funcionamento passo a passo	4
2.3	Estrutura de dados: heap binário de mínimo	5
2.4	Pseudocódigo	6
2.5	Complexidade computacional (O , Ω e Θ)	6
3	Metodologia Experimental	8
3.1	Implementações	8
3.2	Cenários de teste e geração das entradas	8
3.3	Tamanhos avaliados	9
3.4	Protocolo de medição	9
3.5	Ambiente de execução	10
3.6	Resumo dos cenários executados	10
4	Resultados e Análise dos Gráficos	13
4.1	Curvas de tempo por cenário	13
4.2	Comparação entre Python e JavaScript	16
4.3	Trabalho algorítmico: nós expandidos	17
4.4	Validação do modelo teórico	18
5	Aplicabilidade: Eficiência e Limitações	19
5.1	Contextos de eficiência e aplicações	19
5.2	Limitações	20
6	Reflexão Teórica: Classes de Complexidade	21
6.1	O problema pertence à classe P?	21
6.2	Existe uma versão NP do problema?	21
6.3	Problemas semelhantes NP-completos	22
7	Execução do Projeto	23
8	Conclusão	24
	REFERÊNCIAS	25

1 INTRODUÇÃO

O problema do caminho mínimo é central na Teoria da Computação e em inúmeras aplicações práticas. Em contextos onde se dispõe de um destino conhecido e de uma estimativa do custo restante para alcançá-lo, a busca pode ser guiada de forma muito mais eficiente do que na exploração cega. O algoritmo A^* (lê-se “A-estrela”), proposto por Peter Hart, Nils Nilsson e Bertram Raphael em 1968, formaliza essa intuição: ele ordena a fronteira de busca por uma função $f(v) = g(v) + h(v)$, onde $g(v)$ é o custo real acumulado da origem até v e $h(v)$ é uma estimativa heurística do custo de v até o destino. Quando a heurística é admissível — nunca superestima o custo real — o A^* é garantidamente ótimo: o primeiro caminho encontrado é o de menor custo. Quando é também consistente (monótona), cada vértice é expandido no máximo uma vez, eliminando retrabalho.

Este trabalho tem como objetivo estudar o algoritmo A^* sob duas perspectivas complementares. Na perspectiva teórica, descreve-se o que o algoritmo faz, como funciona passo a passo, qual a estrutura de dados empregada e qual a sua complexidade assintótica nas notações O , Ω e Θ . Na perspectiva experimental, o algoritmo foi implementado em duas linguagens de programação — Python e JavaScript (executado no Node.js) — e submetido a um conjunto de experimentos controlados com o objetivo de medir o tempo de execução em três cenários distintos e validar empiricamente o comportamento previsto pela teoria.

O domínio escolhido para os experimentos é uma **grade 2D com obstáculos**: cada célula livre é um vértice, vizinha das células livres adjacentes em quatro direções (cima, baixo, esquerda, direita), com custo unitário por passo. A heurística empregada é a **distância de Manhattan**, $h(v) = |v.linha - destino.linha| + |v.coluna - destino.coluna|$, que é admissível e consistente para movimento em quatro direções com custo unitário — garantindo a otimalidade do algoritmo.

O relatório está organizado da seguinte forma. A Seção 2 apresenta a fundamentação teórica do algoritmo. A Seção 3 detalha a metodologia experimental. A Seção 4 discute os resultados. A Seção 5 aborda a aplicabilidade. A Seção 6 traz a reflexão teórica sobre as classes de complexidade P e NP. A Seção 7 explica como executar os códigos e a Seção 8 conclui.

2 O ALGORITMO A^*

2.1 Definição e objetivo

O algoritmo A^* resolve o problema do **caminho de menor custo entre dois vértices** de um grafo com pesos não negativos nas arestas, utilizando uma **função heurística** que

estima o custo restante até o destino. Enquanto o Dijkstra explora todos os vértices em ordem crescente de distância à origem — equivalente ao A* com $h \equiv 0$ —, o A* “puxa” a busca na direção do destino, expandindo primeiro os vértices com maior potencial de levar a um caminho ótimo. Isso pode reduzir drasticamente o número de vértices processados, especialmente quando a heurística é precisa.

Duas propriedades da heurística são suficientes para garantir a otimalidade:

- Admissibilidade: $h(v) \leq \text{custo_real}(v, \text{destino})$ para todo v . A heurística nunca superestima, portanto o algoritmo nunca descarta prematuramente um caminho ótimo.
- Consistência (monotonicidade): $h(u) \leq \text{custo}(u,v) + h(v)$ para toda aresta (u,v) . Com consistência, cada vértice é expandido no máximo uma vez.

A distância de Manhattan satisfaz ambas as propriedades para movimento em quatro direções com custo unitário: cada passo reduz a distância de Manhattan em exatamente 1 (direção correta) ou aumenta em 1 (perpendicular), portanto a estimativa nunca excede o custo real.

2.2 Funcionamento passo a passo

O funcionamento do A* pode ser resumido nos seguintes passos:

- Inicialização: $g(\text{origem}) \leftarrow 0$; origem inserida na lista aberta com $f = h(\text{origem})$; demais g-scores considerados infinitos.
- Extração do mínimo: remove-se da lista aberta o vértice v com menor f . Se já foi expandido (entrada obsoleta — lazy deletion), descarta-se e continua.
- Verificação do objetivo: se $v = \text{destino}$, retorna o caminho reconstruído pelos pais e $g(v)$.
- Expansão dos vizinhos: para cada vizinho u livre de v , calcula-se $g' = g(v) + 1$. Se $g' < g(u)$, atualiza-se $g(u)$, registra-se v como pai de u e insere-se u na lista aberta com $f = g' + h(u)$.
- Repetição: extração e expansão se repetem até a lista ficar vazia (sem caminho) ou o destino ser extraído.

A **lazy deletion** evita remoções custosas do meio do heap: entradas obsoletas são descartadas ao serem extraídas, preservando a complexidade assintótica e simplificando a implementação.

2.3 Estrutura de dados: heap binário de mínimo

A eficiência do A* depende da estrutura usada para extrair o vértice com menor f .

Utilizou-se uma **fila de prioridade implementada como heap binário de mínimo**, com inserção e remoção em $O(\log n)$.

O critério de desempate possui três níveis: (1) menor $f = g + h$; (2) menor h (vértice mais próximo do destino); (3) menor ordem de inserção (determinismo total). O desempate determinístico é essencial para a comparabilidade entre linguagens.

Em **Python** utilizou-se o módulo `heapq`. Em **JavaScript** desenvolveu-se uma classe `MinHeap` própria (bubble up / sink down). Ambas as implementações usam Map/Set para g-scores, pais e conjunto de expandidos — $O(1)$ médio.

2.4 Pseudocódigo

A seguir, o pseudocódigo do algoritmo implementado:

```
A*(grade, origem, destino):
    aberta ← heap-mínimo ordenado por (f, h, ordem de inserção)
    g[origem] ← 0
    insere origem na aberta com f = h(origem)

    enquanto aberta não vazia:
        (f, h, _, v) ← extrai_minimo(aberta)
        se v já foi expandido: continua      # lazy deletion
        marca v como expandido

        se v = destino:
            retorna (caminho reconstruído pelos pais, g[v])

        para cada vizinho u de v (4 direções, livre, dentro da grade):
            g' ← g[v] + 1
            se g' < g[u]:
                g[u] ← g'
                pai[u] ← v
                hu ← manhattan(u, destino)
                insere u na aberta com f = g' + hu

    retorna "sem caminho"
```

2.5 Complexidade computacional (O , Ω e Θ)

Define-se n como o número total de células ($n = \text{lado}^2$). Cada célula tem no máximo 4 vizinhos ($E = O(n)$); com heap binário, cada operação custa $O(\log n)$.

Limite superior — $O(n \log n)$: no pior caso todos os n vértices passam pelo heap (até 4 inserções por vértice). Custo total $O(n \log n)$. Nenhuma execução ultrapassa esse limite.

Limite inferior — $\Omega(L \cdot \log n)$: o algoritmo precisa expandir cada vértice do caminho ótimo de comprimento L . No melhor caso $L \approx \sqrt{n}$, resultando em $\Omega(\sqrt{n} \cdot \log n)$, empiricamente $\Theta(\sqrt{n})$.

Limite justo — Θ por caso: não existe Θ global, pois os limites superior e inferior não coincidem para todo conjunto de entradas. A análise por caso é:

- Melhor caso (grade vazia, caminho reto): heurística perfeita; A* expande exatamente $\sqrt{n} + 1$ vértices. Tempo $\Theta(\sqrt{n} \cdot \log n)$, empiricamente $\Theta(\sqrt{n})$. Confirmado: 51, 151 e 301 expansões para grades 51×51 , 151×151 e 301×301 .
- Caso médio (obstáculos aleatórios 25%): $\sim 8\%$ das células expandidas, fração aproximadamente constante em n . Tempo $\approx \Theta(n \cdot \log n)$ com constante pequena, empiricamente $\Theta(n)$.
- Pior caso (labirinto em serpentina): $\sim n/2$ vértices expandidos. Tempo $\Theta(n \log n)$. 1.301, 11.401 e 45.601 expansões para $n = 2.601$, 22.801 e 90.601.

Complexidade de espaço: $\Theta(n)$ em todos os casos — g-scores, pais e heap crescem no máximo até n vértices.

3 METODOLOGIA EXPERIMENTAL

3.1 Implementações

O algoritmo foi implementado em duas linguagens: **Python** (python/astar.py) e **JavaScript** no Node.js (javascript/astar.js). Ambas usam heap binário com o mesmo critério de desempate e os mesmos mapeamentos para g-scores, pais e expandidos.

Três mecanismos garantem equivalência: (1) o PRNG **mulberry32**, reproduzido bit a bit nas duas linguagens (aritmética módulo 2^{32}), garante grades idênticas; (2) o mesmo desempate garante expansão dos mesmos vértices na mesma ordem; (3) um script de verificação (tools/verificar_equivalencia.py) confirma via hash FNV-1a. Resultado: nós expandidos e custos são **idênticos** nas duas linguagens em todas as 540 medições.

3.2 Cenários de teste e geração das entradas

Foram definidos três cenários representando os limites assintóticos do algoritmo:

- Melhor caso — grade vazia: sem obstáculos; origem na célula central da primeira coluna, destino na central da última coluna (mesma linha). O A* expande exatamente $L + 1 \approx \sqrt{n} + 1$ vértices do caminho reto, sem explorar nenhum outro.
- Caso médio — obstáculos aleatórios (25%): células bloqueadas com probabilidade 25% usando mulberry32 com semente = número da rodada. Origem no canto superior esquerdo, destino no inferior direito. Se a grade não for conexa, tenta-se a próxima semente.
- Pior caso — labirinto em serpentina: paredes horizontais completas nas linhas ímpares, cada uma com abertura alternando entre extremo direito e esquerdo. A heurística aponta

“para baixo” enquanto o caminho percorre todos os corredores em zigue-zague. Resultado: $\sim n/2$ vértices expandidos.

3.3 Tamanhos avaliados

Cada cenário foi executado em seis tamanhos de lado: **51, 101, 151, 201, 251 e 301** células ($n = 2.601$ a 90.601 células). Os tamanhos nomeados são **pequena (51×51), média (151×151) e grande (301×301)**; os três intermediários aumentam a resolução das curvas.

3.4 Protocolo de medição

Cada combinação de cenário e tamanho foi executada **30 vezes** (540 medições por linguagem). A geração das grades fica fora da cronometragem.

Antes das 30 rodadas, realizaram-se **3 rodadas de aquecimento descartadas** em ambas as linguagens — essencial no Node.js, onde o JIT do V8 otimiza o código nas primeiras execuções.

No caso médio cada rodada usa uma grade diferente (semente = número da rodada), de modo que o desvio-padrão reflete tanto ruído de medição quanto variabilidade real das instâncias.

Funções de alta precisão: Python: `time.perf_counter_ns()` (nanossegundos); JavaScript: `process.hrtime.bigint()`. Ambas cronometram exclusivamente a chamada do algoritmo.

3.5 Ambiente de execução

Especificações da máquina utilizada:

- Processador: Intel Core i5-10500 @ 3,10 GHz (6 núcleos / 12 threads).
- Memória: 16 GB de RAM.
- Sistema operacional: Windows 11 Pro (build 26200).
- Interpretador Python: Python 3.13.7 (CPython).
- Ambiente JavaScript: Node.js v22.21.1 (motor V8).

Os benchmarks foram executados de forma isolada: um cenário por vez, em processo dedicado, sem outras aplicações pesadas em execução concorrente.

3.6 Resumo dos cenários executados

As Tabelas 1, 2 e 3 consolidam as estatísticas obtidas (média $\pm$ desvio-padrão das 30 rodadas, mínimo e máximo) para os três cenários. Dados completos disponíveis em `results/resultados_python.csv` e `results/resultados_javascript.csv`.

Tabela 1 – Tempos de execução – Melhor caso (grade vazia, caminho reto) (ms)

Linguagem	Tamanho	n (células)	Nós exp.	Média ± DP (ms)	Mín (ms)	Máx (ms)
Python	pequena	2.601	51	0,166 ± 0,004	0,160	0,177
JavaScript	pequena	2.601	51	0,261 ± 0,205	0,147	1,215
Python	média	22.801	151	0,704 ± 0,212	0,522	1,089
JavaScript	média	22.801	151	0,379 ± 0,115	0,300	0,716
Python	grande	90.601	301	1,149 ± 0,051	1,125	1,413
JavaScript	grande	90.601	301	0,338 ± 0,067	0,259	0,507

Fonte: elaborado pelos autores (2026).

Tabela 2 – Tempos de execução – Caso médio (obstáculos aleatórios 25%) (ms)

Linguagem	Tamanho	n (células)	Nós exp.	Média ± DP (ms)	Mín (ms)	Máx (ms)
Python	pequena	2.601	~251	0,669 ± 0,203	0,374	1,107
JavaScript	pequena	2.601	~251	0,488 ± 0,168	0,244	0,803
Python	média	22.801	~1.768	5,383 ± 5,982	1,712	26,910
JavaScript	média	22.801	~1.768	1,499 ± 1,943	0,392	8,606
Python	grande	90.601	~6.756	22,101 ± 19,169	6,017	91,736
JavaScript	grande	90.601	~6.756	9,693 ± 6,448	3,021	31,104

Fonte: elaborado pelos autores (2026).

Tabela 3 – Tempos de execução – Pior caso (labirinto em serpentina) (ms)

Linguagem	Tamanho	n (células)	Nós exp.	Média ± DP (ms)	Mín (ms)	Máx (ms)
Python	pequena	2.601	1.301	2,235 ± 0,181	2,115	2,986
JavaScript	pequena	2.601	1.301	0,351 ± 0,105	0,297	0,791
Python	média	22.801	11.401	21,556 ± 0,618	20,700	23,111
JavaScript	média	22.801	11.401	4,292 ± 0,543	3,464	5,827
Python	grande	90.601	45.601	94,109 ± 2,263	90,638	98,685
JavaScript	grande	90.601	45.601	20,874 ± 3,672	16,637	33,033

Fonte: elaborado pelos autores (2026).

4 RESULTADOS E ANÁLISE DOS GRÁFICOS

Esta seção apresenta e interpreta todos os gráficos gerados pelos experimentos (results/graficos/), organizados em quatro grupos: curvas de aderência ao modelo teórico, análise de casos por linguagem, comparação direta Python × JavaScript e trabalho algorítmico pelos nós expandidos.

4.1 Curvas de tempo por cenário

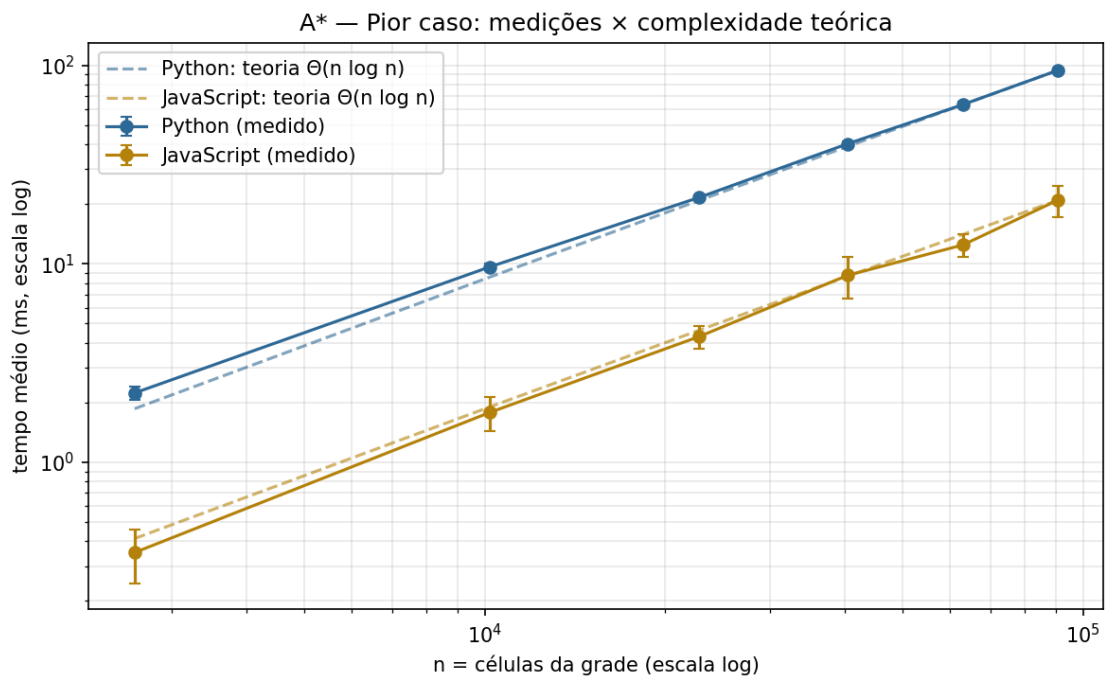


Figura 1 – Aderência ao modelo teórico: pior caso (labirinto em serpentina)

Fonte: elaborado pelos autores (2026).

O gráfico em escala log-log apresenta os tempos medidos de Python e JavaScript no pior caso sobrepostos à curva $c \cdot n \cdot \log_2(n)$. Os pontos de ambas as linguagens acompanham essa reta ao longo de quase duas ordens de grandeza ($n = 2.601$ a 90.601), confirmando $\Theta(n \log n)$. O desvio-padrão é pequeno (Python: $\pm 0,181$ ms a $\pm 2,263$ ms; JS: $\pm 0,105$ ms a $\pm 3,672$ ms), evidenciando medições estáveis. JavaScript é $4,5\times$ mais rápido que Python para $n = 90.601$.

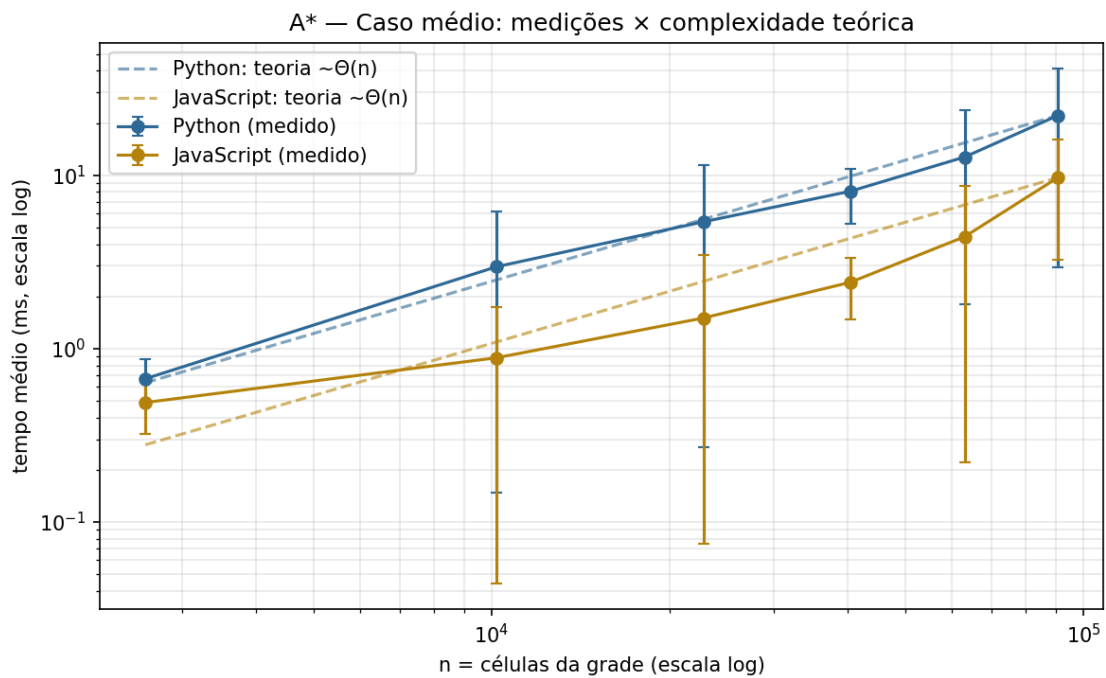


Figura 2 – Aderência ao modelo teórico: caso médio (obstáculos aleatórios 25%)

Fonte: elaborado pelos autores (2026).

Os tempos medidos são sobrepostos à curva $c \cdot n$ (crescimento linear). As barras de erro são consideravelmente maiores do que no pior caso: para $n = 90.601$, o desvio do Python chega a 19,169 ms sobre média de 22,101 ms; o do JavaScript a 6,448 ms sobre 9,693 ms. Esse comportamento é esperado: cada rodada usa uma grade sorteada diferente, e o número de nós expandidos varia entre instâncias. A fração constante de $\sim 8\%$ de células expandidas confirma o crescimento $\approx$ linear.

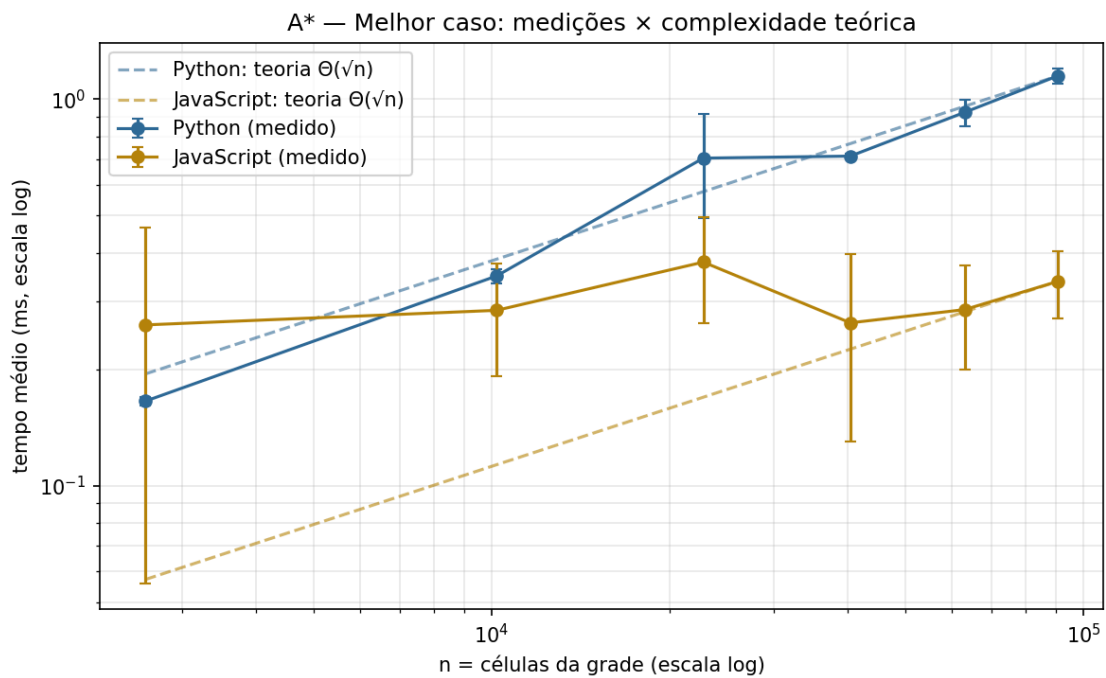


Figura 3 – Aderência ao modelo teórico: melhor caso (grade vazia)

Fonte: elaborado pelos autores (2026).

Os pontos são sobrepostos à curva $c \cdot \sqrt{n}$. O Python segue bem a reta de inclinação 0,5 em escala log-log, confirmando que o número de nós expandidos (exatamente $\sqrt{n}$) domina o tempo. O JavaScript apresenta tempos praticamente constantes ($\sim 0,3$ – $0,4$ ms), pois o custo fixo do motor V8 domina quando apenas 51 a 301 nós são expandidos. Para $n = 2.601$, o Python (0,166 ms) é mais rápido que o JavaScript (0,261 ms); a situação se inverte a partir de $n = 22.801$.

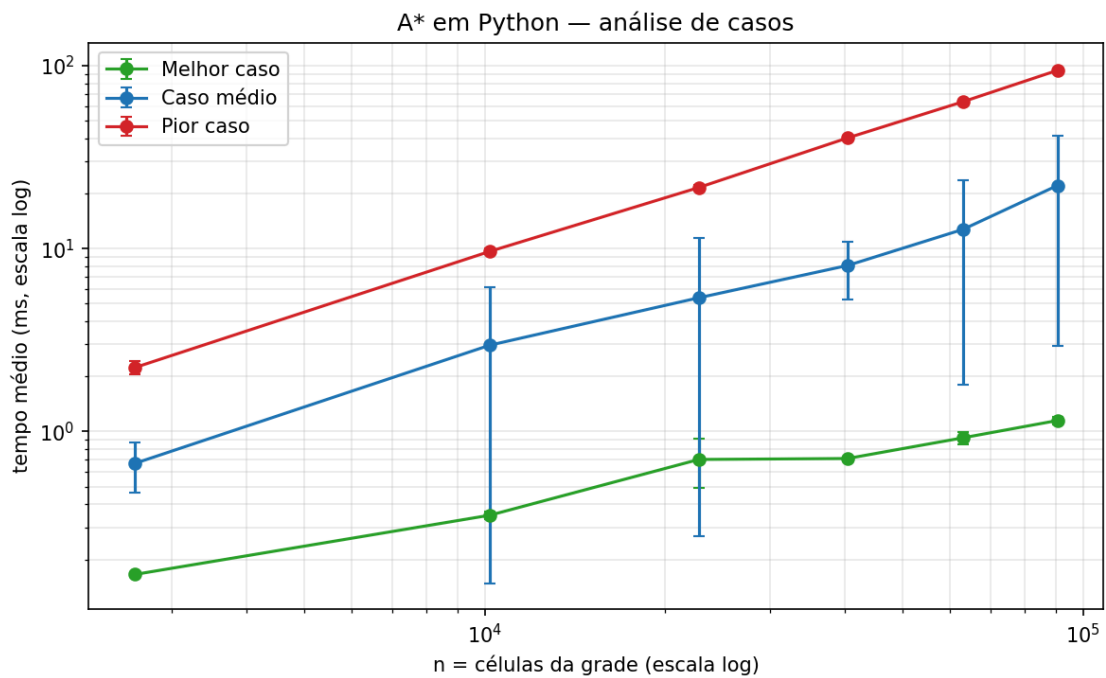


Figura 4 – Análise de casos em Python

Fonte: elaborado pelos autores (2026).

As três curvas (melhor, médio e pior caso) exibem inclinações distintas em escala log-log: $\approx 0,5$ ($\sqrt{n}$), ≈ 1 (linear) e > 1 ($n \log n$). Para $n = 90.601$, o pior caso é 82× mais lento que o melhor caso (94,1 ms contra 1,15 ms). A variabilidade do caso médio é visível pelas barras de erro, enquanto os cenários determinísticos têm barras muito menores.

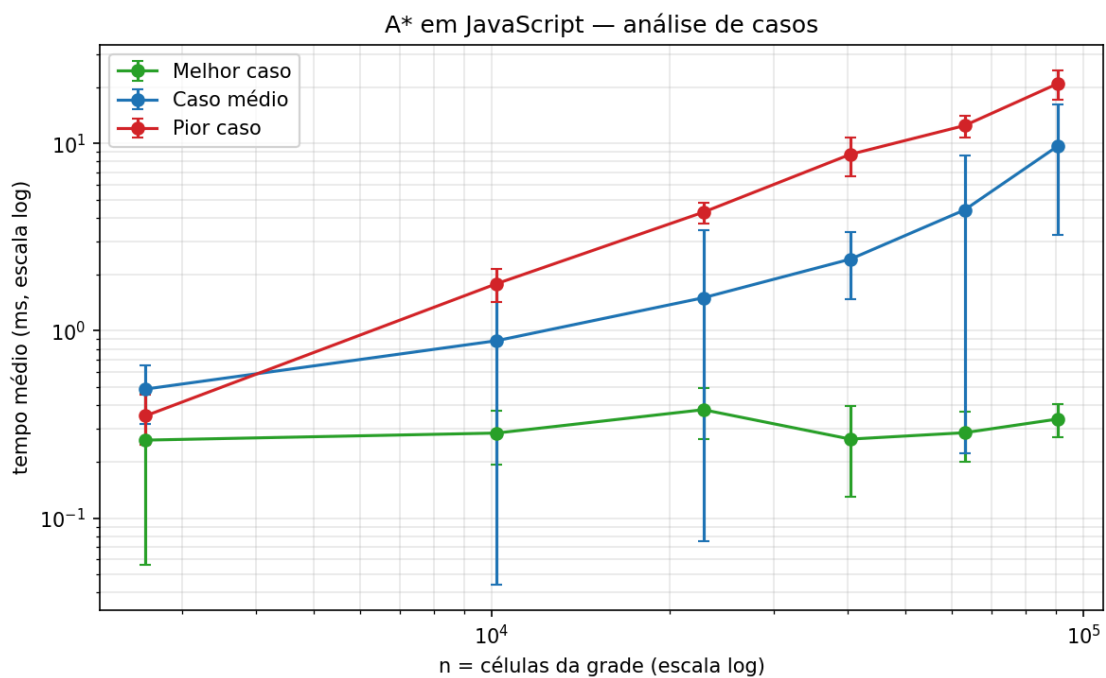


Figura 5 – Análise de casos em JavaScript

Fonte: elaborado pelos autores (2026).

O JavaScript exibe a mesma estrutura de três curvas, mas a curva do melhor caso é praticamente plana. A razão pior/melhor para $n = 90.601$ é $\sim 62\times$ (20,9 ms contra 0,34 ms), ligeiramente menor do que no Python ($82\times$), pois o denominador JavaScript é menor nas entradas grandes. As barras de erro do caso médio também são proporcionalmente grandes.

4.2 Comparação entre Python e JavaScript

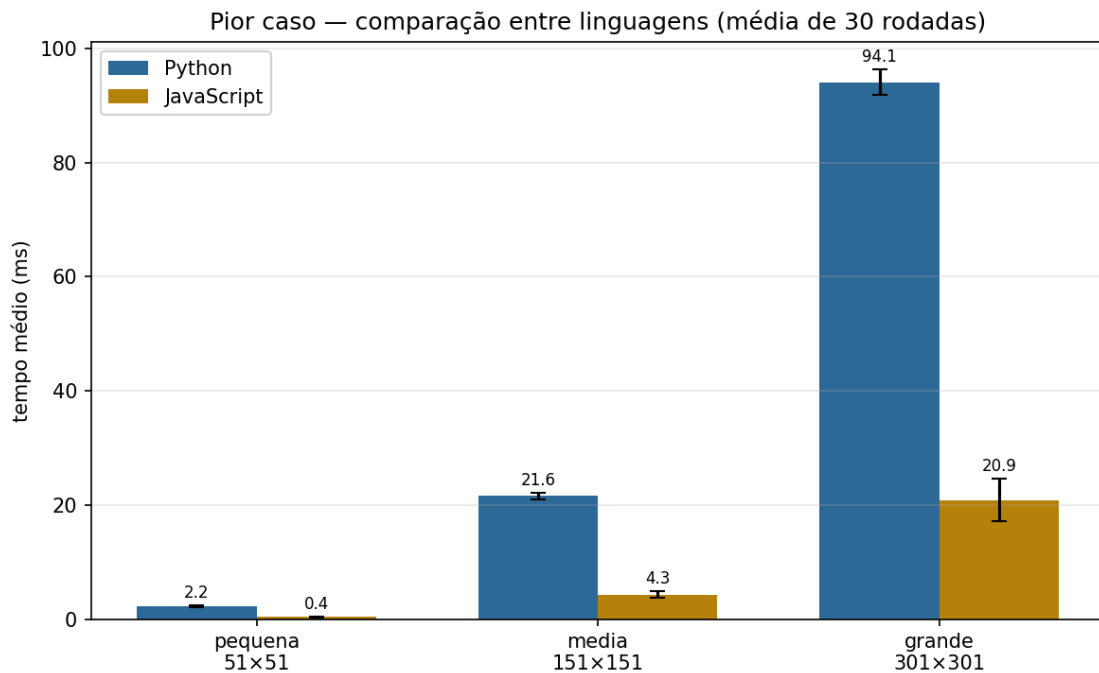


Figura 6 – Comparação direta entre Python e JavaScript (pior caso)

Fonte: elaborado pelos autores (2026).

O gráfico de barras exibe os tempos médios lado a lado para os três tamanhos nomeados. A razão Python/JavaScript vai de $6,4\times$ na grade pequena (2,235 ms contra 0,351 ms) a $4,5\times$ na grade grande (94,109 ms contra 20,874 ms). A diferença reflete a natureza dos runtimes: CPython interpreta bytecodes com despacho dinâmico, enquanto o Node.js (V8) compila para código de máquina com otimizações JIT agressivas. Apesar disso, ambas as implementações exibem o mesmo crescimento assintótico, confirmando que a complexidade é propriedade do algoritmo, não do ambiente de execução.

4.3 Trabalho algorítmico: nós expandidos

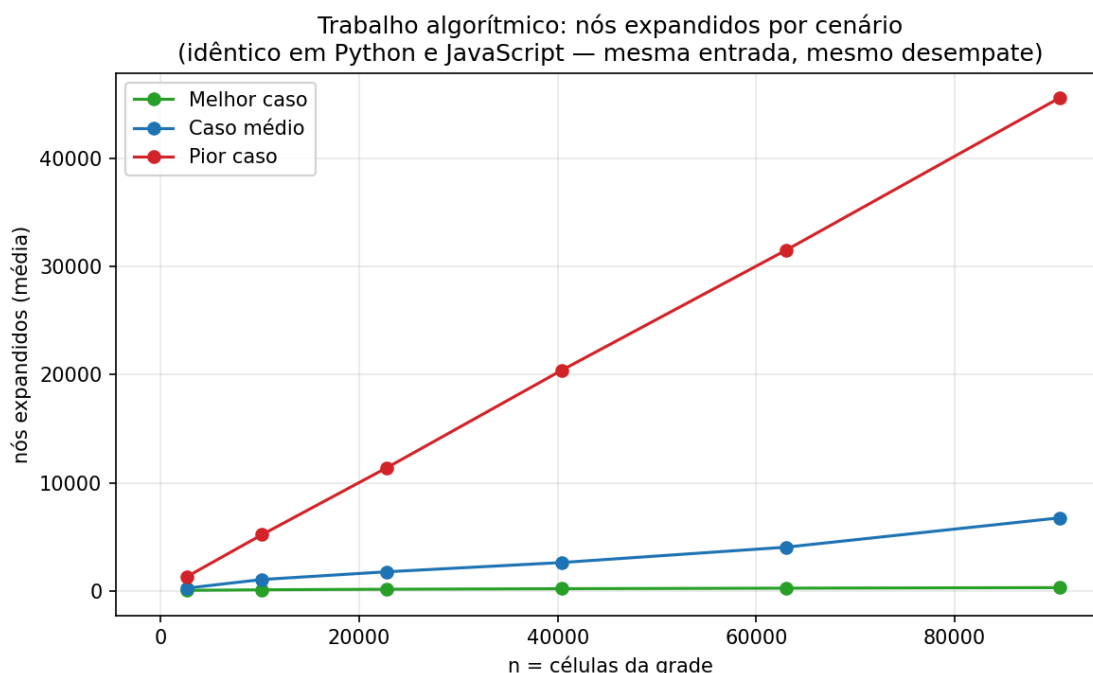


Figura 7 – Nós expandidos por cenário

Fonte: elaborado pelos autores (2026).

Este gráfico mede o trabalho algorítmico puro — nós expandidos — em vez de tempo de CPU. Os valores são idênticos em Python e JavaScript (verificado pelo script de equivalência).

- Melhor caso: proporcional a $\sqrt{n}$ (51, 151, 301 nós). Heurística perfeita; nenhum vértice além do caminho é expandido.
- Caso médio: aproximadamente linear (~ 251 , ~ 1.768 , ~ 6.756 nós, representando $\sim 8\text{--}9\%$ de n). Fração constante expandida.
- Pior caso: proporcional a $n/2$ (1.301, 11.401, 45.601 nós, $\sim 50\%$ de n). O algoritmo percorre todos os vértices do único caminho possível.

4.4 Validação do modelo teórico

A comparação entre os tempos medidos e as curvas teóricas normalizadas (linhas tracejadas nas Figuras 1, 2 e 3) demonstra boa aderência empírica:

- Pior caso: curva $n \log n$ confirmada ao longo de dois fatores de 10 em n ; constante multiplicativa estável para todos os tamanhos.
- Caso médio: reta de inclinação 1 em log-log; maior variância devida à heterogeneidade das instâncias. Fração constante $\sim 8\%$ de nós sustenta $\Theta(n)$.
- Melhor caso: Python segue $\sqrt{n}$ claramente; JavaScript mostra overhead fixo dominante para

n pequeno, convergindo para $\sqrt{n}$ só nas entradas maiores.

5 APLICABILIDADE: EFICIÊNCIA E LIMITAÇÕES

5.1 Contextos de eficiência e aplicações

O A* é o algoritmo de pathfinding mais amplamente utilizado quando: (a) há um único par origem-destino por consulta; (b) os pesos são não negativos; (c) existe heurística admissível barata. Nessas condições, o A* supera o Dijkstra: o melhor caso expande apenas $\sqrt{n}$ vértices, enquanto o Dijkstra exploraria toda a grade (n vértices).

Principais contextos de aplicação:

- Jogos e motores de jogo: pathfinding em grades e navmesh (superfícies 3D). Padrão nas bibliotecas Recast/Detour (Unity, Unreal Engine).
- Robótica e veículos autônomos: planejamento de rota em mapas de ocupação 2D/3D.
- Navegação e GPS: algoritmo ALT (A* with Landmarks and Triangle inequality) e hierarquias de contração para consultas de sub-milissegundo.
- Planejamento automático (IA clássica): puzzles como 15-puzzle e cubo mágico com heurísticas de padrão (pattern databases).
- Redes e telecomunicações: roteamento com estimativas de latência como heurística.

5.2 Limitações

O A* possui limitações relevantes:

- Memória $O(n)$: mantém todos os vértices descobertos em memória. Em espaços exponenciais ($\approx 10^{13}$ estados no 15-puzzle), a fronteira pode esgotar a RAM. Variantes IDA*, SMA* e fringe search trocam memória por recomputação.
- Qualidade da heurística: com $h \equiv 0$ o algoritmo degenera para Dijkstra, explorando n vértices sem benefício heurístico — como no pior caso deste trabalho. Weighted A* explora deliberadamente o trade-off admissibilidade $\times$ velocidade.
- Ambientes dinâmicos: o A* precisa ser reiniciado a cada mudança do mapa. D* e D* Lite foram desenvolvidos para replanejamento incremental eficiente.
- Pesos negativos: assim como o Dijkstra, o A* não funciona com arestas negativas. O Bellman-Ford ($O(V \cdot E)$) é a alternativa.
- Múltiplas consultas: contraction hierarchies ou TRANSIT superam o A* puro quando muitos pares origem-destino precisam ser processados no mesmo grafo.

Em resumo, o A* é altamente eficiente para consulta única em grafo estático com boa heurística, mas há situações — espaços imensos, ambientes dinâmicos ou múltiplas consultas — em que outras abordagens são preferíveis.

6 REFLEXÃO TEÓRICA: CLASSES DE COMPLEXIDADE

Esta seção relaciona o algoritmo estudado com as classes de complexidade P e NP e discute problemas correlatos de difícil solução.

6.1 O problema pertence à classe P?

Sim. A classe P reúne os problemas de decisão resolúveis por algoritmo determinístico em tempo polinomial. O problema do caminho mínimo — versão de decisão: “existe caminho de custo $\leq k$?” — é resolvido pelo A* em $O(n \log n)$, que é polinomial em n . Portanto o problema pertence a P.

Como $P \subseteq NP$, o problema também pertence a NP: dado qualquer caminho proposto, basta percorrê-lo uma vez ($O(n)$) para verificar se o custo é $\leq k$.

6.2 Existe uma versão NP do problema?

O caminho mínimo usual não é NP-difícil, mas variações aparentemente pequenas elevam drasticamente a dificuldade:

- Caminho simples mais longo (longest path): NP-difícil. Trocar “menor” por “maior” elimina a subestrutura ótima que torna o caminho mínimo polinomial.
- Caminho mínimo com restrições de recursos (RCSP): NP-difícil mesmo com pesos positivos. A restrição de orçamento adicional quebra a estrutura polinomial.
- Planejamento multiagente (MAPF): NP-difícil. Encontrar caminhos de custo total mínimo para múltiplos agentes sem colisões não admite solução polinomial geral.
- Problema do (n^2-1) -puzzle ótimo: NP-difícil (Ratner e Warmuth, 1990). O A* é ótimo nesses espaços, mas o número de estados é exponencial — logo o tempo também.

A fronteira entre tratável e intratável é tênue: caminho mínimo em grafos explícitos está em P, mas restrições adicionais, troca do critério de otimização ou exponencialidade do espaço de estados mudam a classe.

6.3 Problemas semelhantes NP-completos

Vários problemas de grafos se assemelham ao caminho mínimo mas são NP-completos:

- Problema do Caixeiro-Viajante (TSP): encontrar o ciclo de menor custo que visita todos os

vértices exatamente uma vez. A versão de decisão (“existe tal ciclo com custo $\leq k$ ”) é NP-completa. A diferença fundamental: o TSP exige caminho hamiltoniano, o que quebra a propriedade de subestrutura ótima.

- Caminho Hamiltoniano: determinar se existe caminho simples passando por todos os vértices exatamente uma vez é NP-completo. O TSP com grafo completo é generalização ponderada.
- Múltiplos agentes com orçamento compartilhado: variantes em que n agentes minimizam custo total com restrição de orçamento são NP-difíceis.

A comparação é instrutiva: o A* em P escala até 90.601 células em < 100 ms, enquanto o TSP e o caminho hamiltoniano não têm solução polinomial conhecida. O fato de o A* pertencer a P é uma vantagem teórica expressiva: eficiência garantida e escalabilidade comprovada, ao contrário dos problemas NP-completos correlatos. Caso fosse descoberto algoritmo polinomial para qualquer NP-completo, ter-se-ia $P = NP$ — uma das maiores questões em aberto da Ciência da Computação.

7 EXECUÇÃO DO PROJETO

Esta seção descreve como reproduzir o experimento a partir do código-fonte.

Repositório: <https://github.com/P4d1lh4/Projeto-Teoria-A->

Estrutura de pastas:

python/	código + benchmark em Python
javascript/	código + benchmark em JavaScript (Node.js)
tools/	script de verificação de equivalência
results/	CSVs e gráficos gerados pelos experimentos
docs/	relatório, slides e tabelas
demo/	demonstração visual interativa (index.html)

7.1 Pré-requisitos

Python 3.10+ e Node.js 18+. Única dependência Python de análise:

```
pip install matplotlib
```

7.2 Executando os testes

```
# Testes unitários Python
python -m unittest discover -s python -v

# Testes JavaScript
node --test javascript/test/astar.test.js

# Verificação de equivalência entre linguagens
python tools/verificar_equivalencia.py
```

7.3 Executando os benchmarks

```
# Benchmark Python → results/resultados_python.csv
python python/benchmark.py
```

```
# Benchmark JavaScript → results/resultados_javascript.csv
node javascript/benchmark.js
```

```
# Modo rápido (5 rodadas, tamanhos nomeados)
python python/benchmark.py --rapido
```

7.4 Gerando os gráficos e tabelas

Após ambos os benchmarks:

```
python python/charts.py
```

Saídas: results/graficos/ (PNGs) e docs/tabelas_resultados.md (tabelas Markdown).

7.5 Demonstração visual

Abrir `demo/index.html` em qualquer navegador exibe o A* explorando a grade em tempo real, com visualização das células abertas, fechadas e do caminho encontrado — útil para a apresentação oral.

8 CONCLUSÃO

Este trabalho apresentou um estudo teórico e experimental do algoritmo A* para o problema do caminho mínimo em grades 2D com obstáculos. Teoricamente, mostrou-se que o algoritmo, implementado com heap binário de mínimo e heurística de Manhattan admissível e consistente, é ótimo, tem limite superior $O(n \log n)$, limite inferior $\Omega(\sqrt{n} \cdot \log n)$ no melhor caso e não admite Θ global. Por ser polinomial, pertence à classe P — ao contrário de variantes correlatas como TSP e caminho hamiltoniano, que são NP-completos.

Os experimentos confirmaram as previsões: no pior caso os tempos crescem como $n \log n$; no caso médio o crescimento é $\approx$ linear com $\sim 8\%$ das células expandidas; no melhor caso exatamente $\sqrt{n}$ vértices são expandidos. Python segue $\sqrt{n}$ claramente; JavaScript exibe overhead fixo dominante para n pequeno.

Na comparação entre linguagens, JavaScript (Node.js/V8) foi de $4,5\times$ a $6,4\times$ mais rápido no pior caso graças ao JIT. Python apresentou menor variância nos cenários determinísticos e foi mais eficiente para entradas muito pequenas no melhor caso. Ambas as implementações exibiram o mesmo padrão assintótico, confirmando que complexidade é propriedade do algoritmo, não da linguagem.

A verificação formal de equivalência — mesmo PRNG bit a bit, mesmo critério de desempate e script FNV-1a — garantiu que os dois runtimes processaram exatamente as mesmas entradas e expandiram exatamente os mesmos nós, tornando a comparação de tempos uma medição pura de desempenho de runtime, sem interferência algorítmica.

REFERÊNCIAS

- HART, P. E.; NILSSON, N. J.; RAPHAEL, B. A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, v. 4, n. 2, p. 100–107, 1968.
- RUSSELL, S.; NORVIG, P. Artificial Intelligence: A Modern Approach. 4. ed. Hoboken: Pearson, 2020. (Capítulo 3 — busca informada e exploração)
- CORMEN, T. H. et al. Introduction to Algorithms. 4. ed. Cambridge: MIT Press, 2022. (Capítulo 22 — Dijkstra e filas de prioridade)
- RATNER, D.; WARMUTH, M. The (n^2-1) -puzzle and related relocation problems. Journal of Symbolic Computation, v. 10, n. 2, p. 111–137, 1990.
- GAREY, M. R.; JOHNSON, D. S. Computers and Intractability: A Guide to the Theory of NP-Completeness. New York: W. H. Freeman, 1979.
- ZIVIANI, N. Projeto de Algoritmos com Implementações em Java e C++. São Paulo: Cengage Learning, 2007.